

KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER AND ELECTRICAL ENGINEERING

BSc. Computer Engineering

**WEB BASED INTERNSHIP AND ACADEMIC REPORT MANAGEMENT
SYSTEM**

STUDENT DETAILS

Tetty Ara Dede-1828722

Tsidi Carl Etornam Kwame-1829022

Peter Saadade Nyarko-1825722

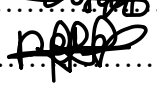
Supervised By: Dr. (Mrs.) Theresa-Samuella Maame Atwemaah Adjaidoo.

MAY 2026

DECLARATION

We, the undersigned student project team, hereby declare that we have discussed and agreed to undertake the project titled “**Web Based Internship and Academic Report Management System**” under the supervision and guidance of Dr. (Mrs.) Theresa-Samuella Maame Atwemaah Adjaidoo. We understand the scope of the work and commit to executing the project to the best of our abilities.

Student Team Members:

1. Tettey Ara Dede 
2. Tsidi Carl Etornam Kwame..... 
3. Nyarko Peter Saadade 

Supervisor: I Dr. (Mrs.) Theresa-Samuella Maame Atwemaah Adjaidoo, hereby confirm my consent to supervise this project and guide the team throughout its life cycle.



.....

[Supervisor’s Signature & Date]

ACKNOWLEDGEMENTS

Our thanks go first and foremost to our Heavenly Father who has seen us through our academic journey up to this point and for all His grace and mercies in the undertaking of this project.

We would also like to thank and acknowledge our supervisor, Dr. Theresa S.M.A Adjaidoo, for her dedication to us even long before this project. Her guidance and criticism and motivation have greatly shaped our academic journey the project to its current state.

We are grateful to all the lecturers department of computer engineering for their genuine efforts to impart their knowledge onto us in order to equip us with the valuable skills which have greatly benefited us in this project.

Finally, we thank our colleagues and friends for their encouragement and useful insights.

DEDICATION

This work is dedicated to our parents who have worked tirelessly and sacrificed a lot to give us such a great opportunity to study in KNUST for four years.

Abstract

This project presents the design and development of a Web-Based Internship and Academic Report Management System aimed at improving internship management activities, weekly log submissions and report generation in tertiary institutions. The study shows the challenges associated with the traditional paper-based internship management systems, such as inefficient communication, delayed feedback, document misplacement and sub-standard student reports. The proposed system provides a centralized digital platform that helps students to register internship placements, upload weekly logs, generate internship reports and receive supervisor feedback in a structured and efficient manner.

The system was developed using the Scrum Agile software development methodology to support iterative development and continuous feedback throughout the project lifecycle. The frontend was implemented using React, Vite, and Tailwind CSS to provide a responsive and user-friendly interface while Django was used for backend development and PostgreSQL for database management. The system also incorporates Large Language Model (LLM) integration to support automated report generation from structured internship logs, thereby improving report consistency and academic quality.

The architecture of the system follows a client-server model integrated within the Microsoft Teams environment to improve accessibility and collaboration between students and supervisors. Core functionalities include user authentication, internship registration, log submission, automated report generation, supervisor review and feedback and centralized data management.

The project aims to demonstrate the practical application of modern web technologies and artificial intelligence techniques in solving issues associated with internship and academic report management systems.

Contents

DECLARATION	2
ACKNOWLEDGEMENTS	3
DEDICATION	4
Abstract	5
Contents	6
Table of Figures	7
Table of Abbreviations	8
1 CHAPTER ONE: INTRODUCTION	9
1.1 Background of Study	9
1.2 Problem Statement	10
1.3 Objectives	10
1.3.1 General Objectives	10
1.3.2 Specific Objectives	10
1.4 Significance of Study	11
1.5 Scope of Report	11
2 CHAPTER TWO: LITERATURE REVIEW	13
2.1 Introduction	13
2.2 Similar Systems	13
2.2.1 InPlace Smart Placement Solutions	13
2.2.2 PebblePad	14
2.3 Large Language Models for Automated Report Generation	15
2.3.1 Overview of Large Language Models	15
2.3.2 Capabilities of LLMs in Report Generation	16
2.3.3 Prompt Engineering for Structured Report Generation	16
2.3.4 API-Based Integration of Large Language Models	16
2.3.5 Controlling LLM Output for Academic Reports	17
2.3.6 Challenges of Using Large Language Models	17
2.3.7 LLMs in Automated Writing and Documentation Systems	17
2.3.8 Justification for LLM Integration in Automated Report Generation	18

3	CHAPTER THREE: RESEARCH METHODOLOGY	19
3.1	Introduction.....	19
3.2	Software Development Life Cycle.....	19
3.2.1	Software Development Model: Scrum Agile Model	20
3.3	Overall Description.....	21
3.4	Software System Design	21
3.4.1	System Architectural Style	22
3.5	Logical Design.....	23
3.5.1	Data Flow.....	23
3.6	Core Architecture.....	24
3.6.1	Connections.....	24
3.6.2	Entity Relationship Diagram (ERD).....	24
3.6.3	Users and Services	27
3.6.4	Dependencies and Languages	28
3.7	THE USER INTERFACE.....	30
4	References.....	37

Table of Figures

Figure 1.	QuantumIT Logo	13
Figure 2.	Inplace Student Placement Dashboard	13
Figure 3.	PebblePad Logo.....	14
Figure 4.	Scrum Development Cycle [16]	20
Figure 5.	Client-Side Design. [17].....	22
Figure 6.	Server-Side Design [17]	22
Figure 7.	Data Block Diagram	23
Figure 8.	Entity Relationship Diagram	26
Figure 9.	Use-Case Diagram.....	27
Figure 10.	Login Page.....	30
Figure 11.	Student Dashboard.....	31
Figure 12.	Internship Registration	31
Figure 13.	Upload Weekly Logs.....	32

Figure 14. Feedback Page	33
Figure 15. Profile Page	33
Figure 16. Supervisor Dashboard	34
Figure 17. Supervisor’s Weekly Reports Page	35
Figure 18. Supervisor’s feedback page	35

Table of Abbreviations

NLP	Natural Language Processing
API	Application Programming Interface
LLM	Large Language Model
SDLC	Software Development Life Cycle
IDE	Integrated Development Environment
VSCode	Visual Studio Code

1 CHAPTER ONE: INTRODUCTION

1.1 Background of Study

Advancement in technology has greatly transformed the way academic institutions manage academic and administrative processes. Many universities have integrated digital solutions to simplify operations such as student and course registration as well as result processing [1]. Despite this advancement in educational technology, the management of student internship programs, submission and review of internship logs and reports has continued to lag in the race towards digitization. These workflows have remained mostly manual and rely on paper-based procedures which are prone to errors, difficult to coordinate and sometimes inefficient.

Internship programs also known as industrial attachments bridge the gap between academic learning and professional practice. They give students the opportunity to apply theoretical knowledge in real-world environments while developing professional skills [2]. In Kwame Nkrumah University of Science and Technology, industrial attachment is a requirement for completion of undergraduate programs. Students secure placements for their attachment and spend a minimum of 4 weeks understanding the professional setting to submit log sheets and a report for evaluation from academic supervisors [3]. However, ensuring that internship programs run smoothly involves multiple interactions among students, academic supervisors, industry-based supervisors and faculty coordinators. Managing these interactions and internship processes through physical documentation and informal communication channels introduces challenges such as misplacement of documents, difficulty in monitoring large numbers of students efficiently and delayed feedback.

Aside from these administrative challenges, another major concern is the low quality of student-submitted internship reports. Majority of the students struggle to meet the required academic standards in their reports, submitting reports that lack the proper structure, correct referencing and in-depth analysis of subject matter. This issue is due to insufficient guidance during report writing, where students are left to work on their own with no iterative feedback from their supervisors before final submission. As a result, large volumes of reports submitted are below standard and ultimately produce poor academic accuracy of the institution's internship program.

1.2 Problem Statement

The management of internship programs and report submissions at most tertiary institutions in Ghana is handled through manual, paper-based processes that are inefficient and difficult to monitor. Students, faculty supervisors and departmental coordinators lack a unified platform to communicate, submit documents and track progress resulting in delayed feedback and misplaced documents. The nature of this manual approach also means that students receive no iterative guidance during the report writing process, contributing to a high volume of submissions falling below the required academic standard.

1.3 Objectives

1.3.1 General Objectives

To design and implement a web-based management system that automates and centralizes student internship log sheet and report submission.

1.3.2 Specific Objectives

- To review existing literature on web-based academic management systems and role-based access control frameworks to identify best design practices and system requirements.
- To design the system architecture, database schema and user interface prototypes for a multi-role web application serving students and supervisors.
- To develop and integrate core modules for user authentication, internship registration and tracking, report submission and supervisor review and grading.
- To test and validate the system's functionality and usability.

1.4 Significance of Study

The development of a Web-Based Internship and Academic Report Management System greatly benefits students, faculty supervisors and the institution at large.

Students will directly benefit from the system, since it will provide them with a structured and accessible platform to generate and submit reports and receive timely feedback.

Faculty supervisors will not be burdened to manually collect and track student logs and reports. The system will also allow them to manage multiple students efficiently and complete assessments within a centralized platform, reducing the possibility of oversights.

The institution will have a structured and organized internship management system. The system will help uphold the academic integrity of the internship program, reduce the risk of document loss and reflect the position of the institution as one that embraces digital innovation in its academic administration.

1.5 Scope of Report

This report covers the design and development of a web-based Internship and Academic Report Management System. The report is organized into the following chapters:

Chapter Two examines existing systems and research related to academic report submission and web-based information systems. It further explores the role of Large Language Models (LLMs) in automated report generation and their potential integration into academic systems, identifying the gaps in existing solutions that this project seeks to address.

Chapter Three covers the methodology used in the design and development of the system. It talks about the software development lifecycle and justifies the selection of the Scrum Agile model for the development approach. The chapter further provides an overall description of the system, its architectural style, logical design, data flow, and core architecture.

Chapter Four centers on the testing and evaluation of the system. It indicates the testing strategies used and evaluates the extent to which the system meets objectives.

Chapter Five concludes the report and key challenges faced during the lifetime of the project are highlighted. Suggestions are also made for future improvements of the study.

2 CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter highlights the existing softwares from which much inspiration was drawn for our project as well as the key technology that makes our system unique.

2.2 Similar Systems

2.2.1 InPlace Smart Placement Solutions



Figure 1. QuantumIT Logo

Placement Management					
View and manage placements for student field experiences.					
PLACEMENTS PARTNER SITES MENTORS IMPORTS					
Placements					
Search Search by student name, email or placement ID Filter by All Course Codes Term1					
STUDENT ↑	COURSE CODE	TERM	PLACEMENT ID	SITE	MENTOR
Deborah, student studentsfpprod4124@yopm...	English Course	Term1	1145765857	-	-
Elizabeth, student studentsfpprod4104@yopm...	Placement Course	Term1	8656338506	Santa Catalina School-updated	-

Figure 2. Inplace Student Placement Dashboard

InPlace Network is a web-based placement management system developed by QuantumIT, designed to manage student work placements, internships, and experiential learning programs for educational institutions. It streamlines communication among students, employers, and universities, improving coordination, compliance, and administrative efficiency.

It was developed by QuantumIT and released in the early 2010s for Universities and higher educational institutes with the primary purpose of enabling the management of students while they undertook internships. Students are able to upload structured logs of their activities during their internships which their supervisors can assess, validate and comment on.

InPlace features automated reporting capabilities that generate structured reports based on predefined templates. Based on the structured nature of the logs, report compilation just assembles data from the logs for report compilation. The main shortcoming of this InPlace is the incapability of understanding log context in order for summarization or report generation from logs that do not follow the predefined structure set by the system.

The system is widely adopted across universities in Australia, the United Kingdom, and North America. It supports faculties such as health, education, and engineering where professional placements are integral to accreditation. Integration with institutional systems like learning management and student information platforms helps automate data flow and reduce administrative workload. [4]

2.2.2 PebblePad



Figure 3. PebblePad Logo

PebblePad is a digital portfolio and learning system that provides a platform for uploading one's learning journey, amongst other activities. Developed by Pebble Learning in 2004, it's used by over 150 institutions worldwide to support out-of-classroom learning. Students are provided with

structured templates known as workbooks. These guide the users to upload daily or weekly logs of activities and reflections together with supporting images. These logs can be documentations of their learning journey, their experiences during industrial training and other personal skill acquisition and academic endeavours.

The system also provides report compilation capability where students can compile their logs and reflections into a single document based on customized templates.

PebblePad also supports supervisor feedback and assessment using predefined rubrics, feedback templates, scorecards and more. [5]

Other similar software include Jasper AI and Symplicity.

2.3 Large Language Models for Automated Report Generation

2.3.1 Overview of Large Language Models

Large Language Models (LLMs) are sophisticated deep learning algorithms trained on huge datasets allowing them to understand and generate natural language. This technology is made possible due to years of research into natural language processing (NLP) and machine learning. The majority of existing LLMs are based on a type of neural network called the Transformer Architecture. This architecture enables the LLMs to understand and generate massive amounts of text by utilizing self-attention models to detect dependencies between elements of long sequences [6]. Similar to how humans, understand sentences based on context, self-attention models are a mathematical way of assigning context weights to tokens.

LLMs undergo pre-training on massive datasets sources from the internet, news articles and other publicly available sources of information. The massive text gotten from these sources are then broken down into tokens for easier processing. Fine-tuning is then done using smaller and more specific datasets. The pre-trained LLM can be thought of as a general purpose processor whereas the fine-tuned LLM would be an application specific processor. Fine-tuning can enable specialization in different tasks such as summarization, paraphrasing, question answering, generation of complex text and even image generation[7]. Examples of these LLMs include open ai's GPT models, anthropic's sonnet and opus models, as well as meta's llama models. Within the domain of report generation, LLMs demonstrate certain advantages by converting well-organized user inputs into narrative texts, which enables the generation of multi-section reports[8].

2.3.2 Capabilities of LLMs in Report Generation

Some significant capabilities can be mentioned for LLMs in regard to report generation. Due to their text generation capabilities, LLMs can write paragraphs whole paragraphs when prompted to. This is observed even in our day to day usage of LLM's through their public interfaces such as chatgpt, claude ai and gemini. Thus, it is no surprise that they are capable of reports containing multiple sections including background information, objectives, and summary [7, 9].

Regarding their summarization capabilities, LLMs enable the reduction of a vast amount of text into brief sentences, which will be useful for creating a brief overview of daily task logs in each week. Another important feature is the ability to paraphrase and rephrase provided text in an academic style.

2.3.3 Prompt Engineering for Structured Report Generation

Prompt engineering is a key element in shaping the outputs of LLMs. The process involves formulating structured inputs that enable the LLMs to generate the intended outcomes. The formulating involves giving as much specific information about the desired output as possible, whereby we include context and examples. For example, if you instructed an LLM to give you a cheese-filled tomato spaghetti recipe as opposed to simply asking for a pasta recipe, you would find the first prompt to be more precise in providing the output you require. This process works just like commissioning an artist, whereby the less details you specify, the more creative freedom the artist has whereas, the more details you specify the more the final output is influenced by the you.

There are different types of prompting techniques that can be used when needed. These include, zero-shot prompting, few shot prompting and role prompting. Zero-shot prompting involves simply instructing the LLM on the task to perform without any additional context or examples. Few-shot prompting is an upgrade of zero-shot due to the provision of examples or baselines that must be adhered to. This serves a viable approach to the report generation system proposed by us given the fact the provided examples reduce the 'creativity' of the models when generating the outputs. These formulated structured prompts enhance consistency in the outputs produced by LLMs. In role prompting, just as the name suggests, the LLM is assigned a role that governs the style in which it generates text. The LLM could be prompted to give outputs in a formal tone, informal tone, as a medical professional, as an academic and whole lot of others. [10]

2.3.4 API-Based Integration of Large Language Models

Most current LLMs can be accessed via cloud APIs. By connecting to an LLM API, software applications can submit prompts to the LLM and get back the generated responses. Such a method helps to simplify deployment and decrease computational power demands [11].

In the proposed system design, the prompt would be generated on the server side based on user inputs, after which it would be submitted to the LLM API. After processing the request, the LLM will generate the text and return it back to the server, where the output will be processed and then sent to the frontend for display.

2.3.5 Controlling LLM Output for Academic Reports

When interacting with LLMs through their APIs, certain settings help in arriving at the desired outputs. These settings include sampling parameters such as temperature and top-p/top-k. They determine the randomness of generated text, with lower values producing more deterministic and factual outputs. These settings are intrinsic to the model and therefore can be provided through the API or not. For academic report generation, low temperature values are preferred to ensure consistent formatting and reduce creative deviations [10].

Additionally, output length limits can be specified to ensure each generated section meets report requirements. Structured prompts combined with controlled sampling improve reliability and maintain academic tone in generated reports.

2.3.6 Challenges of Using Large Language Models

While large language models (LLMs) prove to be quite effective, there are several challenges associated with them.

Firstly, hallucinations occur when the LLM creates inaccurate or misleading information. This happens because the model tends to derive information based on learned patterns instead of facts [9]. In report generation, the problem may be overcome through the use of prompts containing information provided by users and correct sampling parameters [12].

Second, since LLMs rely on probabilistic sampling, multiple prompts may lead to slightly different results, which may be avoided through deterministic generation.

Furthermore, using LLM-based services may incur costs and delays due to API calls, which could be lowered by optimizing the prompts to use fewer tokens.

2.3.7 LLMs in Automated Writing and Documentation Systems

LLMs have been used widely in writing assistants and document generation platforms. For instance, Jasper AI, ChatGPT, and Notion AI, leverage LLMs to create structured documents, such as reports, summaries, and documentation [13].

Even so, these general-purpose tools do not offer custom templates for internship report generation. On the other hand, the automated report generator utilizes prompts with structural templates and user inputs to create structured academic reports

2.3.8 Justification for LLM Integration in Automated Report Generation

LLMs are capable of producing structured documents, as evidenced by their ability to recognize contextual information, adhere to prompts, and organize the output text [7], [9]. Thus, LLMs could be used to develop an automated report generator system. Since LLMs could be integrated through APIs, incorporating them into existing web applications would become easier [11]. Overall, the use of LLMs is expected to provide automatic internship report generation services using structured prompts and user inputs.

3 CHAPTER THREE: RESEARCH METHODOLOGY

3.1 Introduction

This research methodology explains the step-by-step approach used to build the system. The goal is to make sure the system is built well, is easy to update, and meets what users need by using careful, repeated development and testing.

3.2 Software Development Life Cycle

The Software Development Life Cycle (SDLC) is a structured process that defines the stages a software goes through from the initial stage to the deployment stage. This cycle reduces risk and enhance quality at the end of the project.[14]

However, there are many software processes, and they must contain these four fundamental activities;

- Software specification: The functionality or requirements of the software and constraints on its operation must be defined.
- Software development: The software must meet the specifications of what is to be produced.
- Software testing and validation: The software must be validated and tested to ensure that it does what the user wants.
- Software maintenance and improvement: The software must undergo maintenance and improvements to meet the changing needs of users.[14]

3.2.1 Software Development Model: Scrum Agile Model

For this project, we chose the Agile software development model. We chose this model because it welcomes changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage. [15]

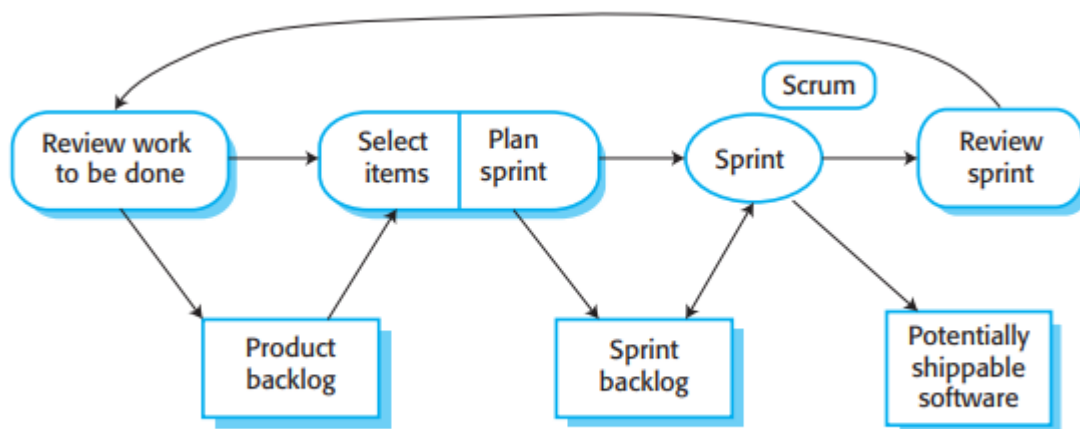


Figure 4. Scrum Development Cycle [16]

Scrum is better than other ways of building software because it makes sure that working features are checked often to lower the chance of creating a system that does not meet what users want.

Three benefits of Scrum

- Scrum allows changes to be made in any stage of development, making it ideal for complex and dynamic projects.
- They undergo regular reviews to provide feedback, ensuring the system aligns with user requirements.
- It encourages collaboration by assigning shared responsibilities and holding regular meetings to track progress.[16]

3.3 Overall Description

This system is designed to track the process from gaining internship, submitting internship weekly logs to report submissions. It helps students to upload weekly logs, which are then reviewed and evaluated by their supervisor. These activities are managed by an administrator.

The system will be developed using React, Vite, and Tailwind CSS for the frontend which ensures that the user has a responsive and interactive interface. The backend will be developed using Django to enable a fast and secure web application. PostgreSQL will be used for the database because it is reliable. Version control will be managed using GitHub while Vscode will serve as the integrated development environment (IDE).

3.4 Software System Design

This system design involves operations including data flow, user roles and the processes that go on in the system that describes how the system components work together.

3.4.1 System Architectural Style

This project uses a client-server. Client-server architecture is a widely used system design where multiple clients request services or resources from a central server. The server manages processing, data storage, and resources, while clients handle user interaction.[17]

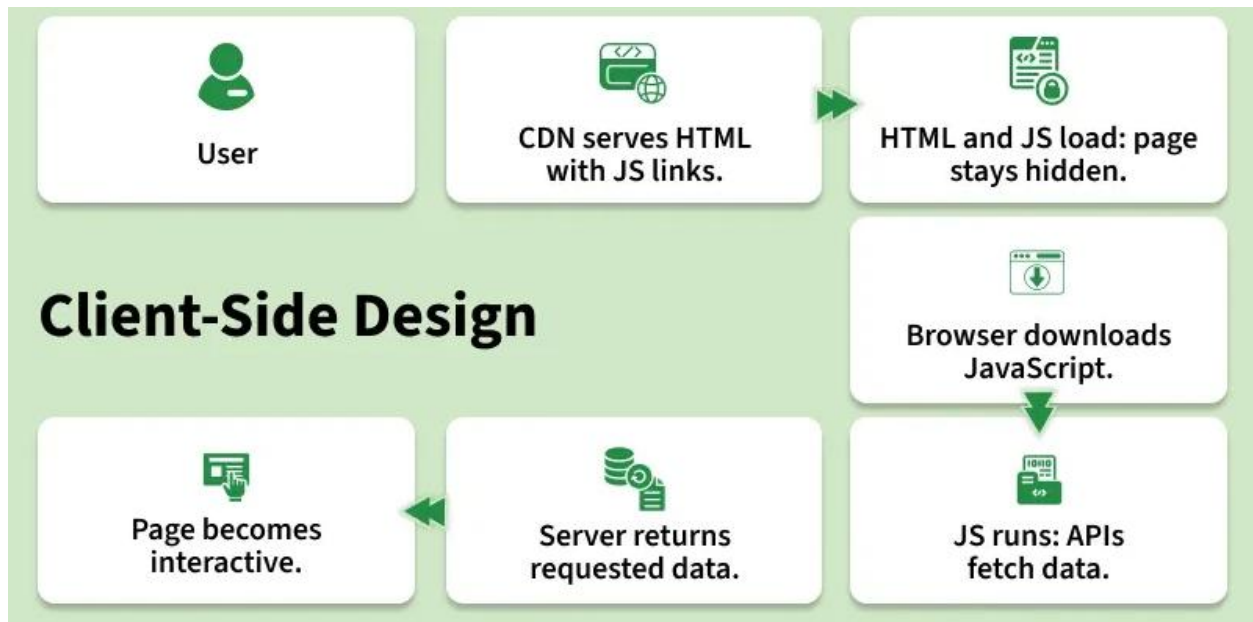


Figure 5. Client-Side Design. [17]

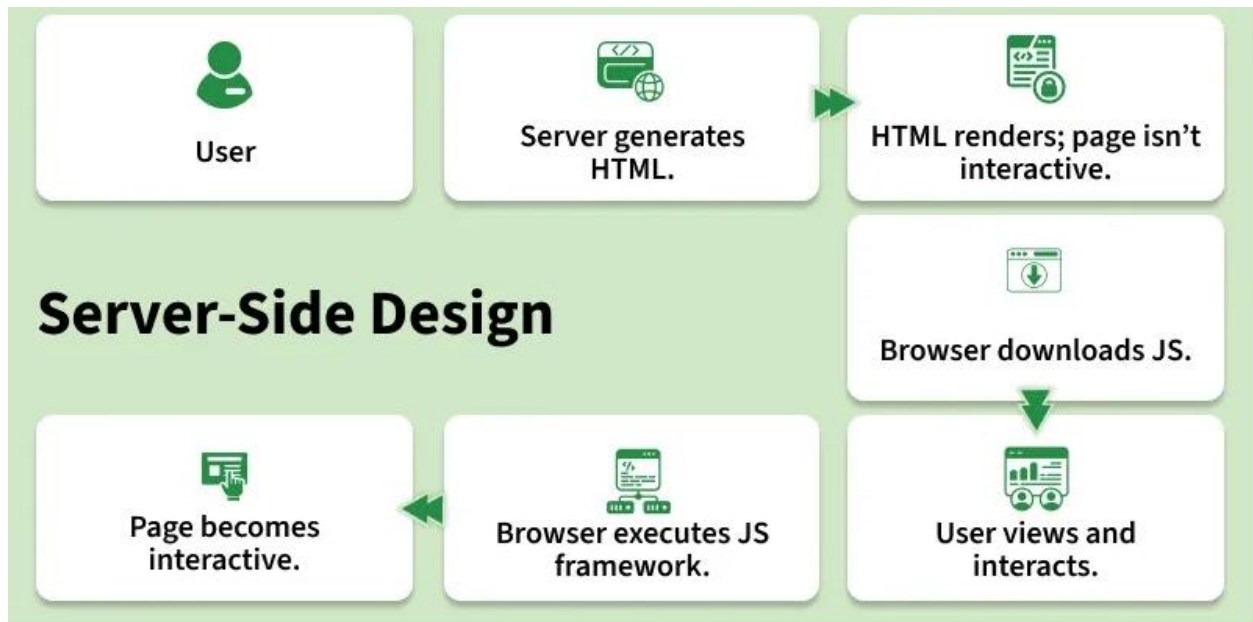


Figure 6. Server-Side Design [17]

3.5 Logical Design

The logical design describes how the components in the system interact with each other.

3.5.1 Data Flow

The data basically describes how user requests are being carried out from the interaction with the frontend to initiate a request, how the request is processed to obtain the result which are then displayed to the user on the frontend.

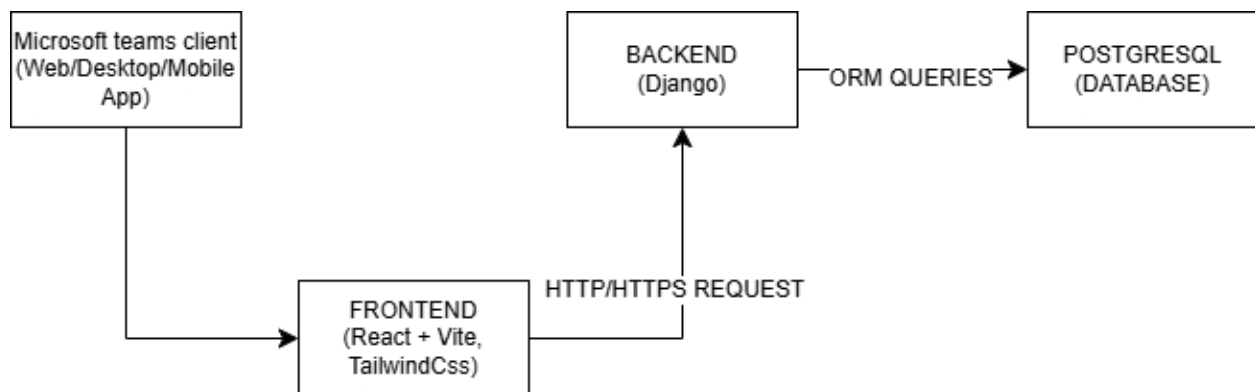


Figure 7. Data Block Diagram

The system is a web-based application which is integrated in Microsoft teams as a plugin. Which allows students and supervisors to gain access to the internship management system in the Microsoft Teams environment.

The users interact with the frontend which sends http/https requests using RESTful APIs to the backend. The backend then interacts with the PostgreSQL database, where all the system information or data is stored including users, internships, weekly logs, reports and evaluations is stored and managed. The backend fetches the data and displays it to the user with the frontend.

3.6 Core Architecture

The core architecture is a web-based application that includes a frontend, backend, database layer and integration layer within Microsoft Teams.

The frontend layer uses react to provide a dynamic user interaction, and Vite for fast deployment with Tailwind CSS for responsiveness.

The backend layer uses Django that manages the server-side logic and API endpoints.

The database layer uses PostgreSQL that stores all the system data with strong relational integrity.

The integration with Microsoft Teams allows the web app to function as a plugin.

3.6.1 Connections

The system components interact through secure HTTP/HTTPS using RESTful APIs. Data is exchanged in JSON format to ensure compatibility between the back and frontends.

3.6.2 Entity Relationship Diagram (ERD)

The entity relationship diagram shows how the various key entities are stored and how they relate with each other.

- Students: This represents all the students in the database. It is the key entity in the system. It serves as the pillar for all other entities. Every internship, log, supervisor and report are tied to a specific student. It uses a primary key “Student_id”.
- Supervisor: This represents all the supervisors in the database. A supervisor can have many students and can write many reviews. It uses a primary key “supervisor_id”
- Report: This represents where the generated reports are stored. A student can have only one report and a report can have one review. Its primary key is denoted by Report_id.

- Log: This entity represents the log sheets that will be submitted by the students. A student can have many log sheets and a log sheet can have one review. Its primary key is denoted by log_id.
- Internship: This entity represents the internships of the students. One student can only have one internship. Its primary key is denoted by internship_id.
- Reviews: This represents the review entity which stores the feedback from the supervisors to various students. Its primary key is denoted by review_id.

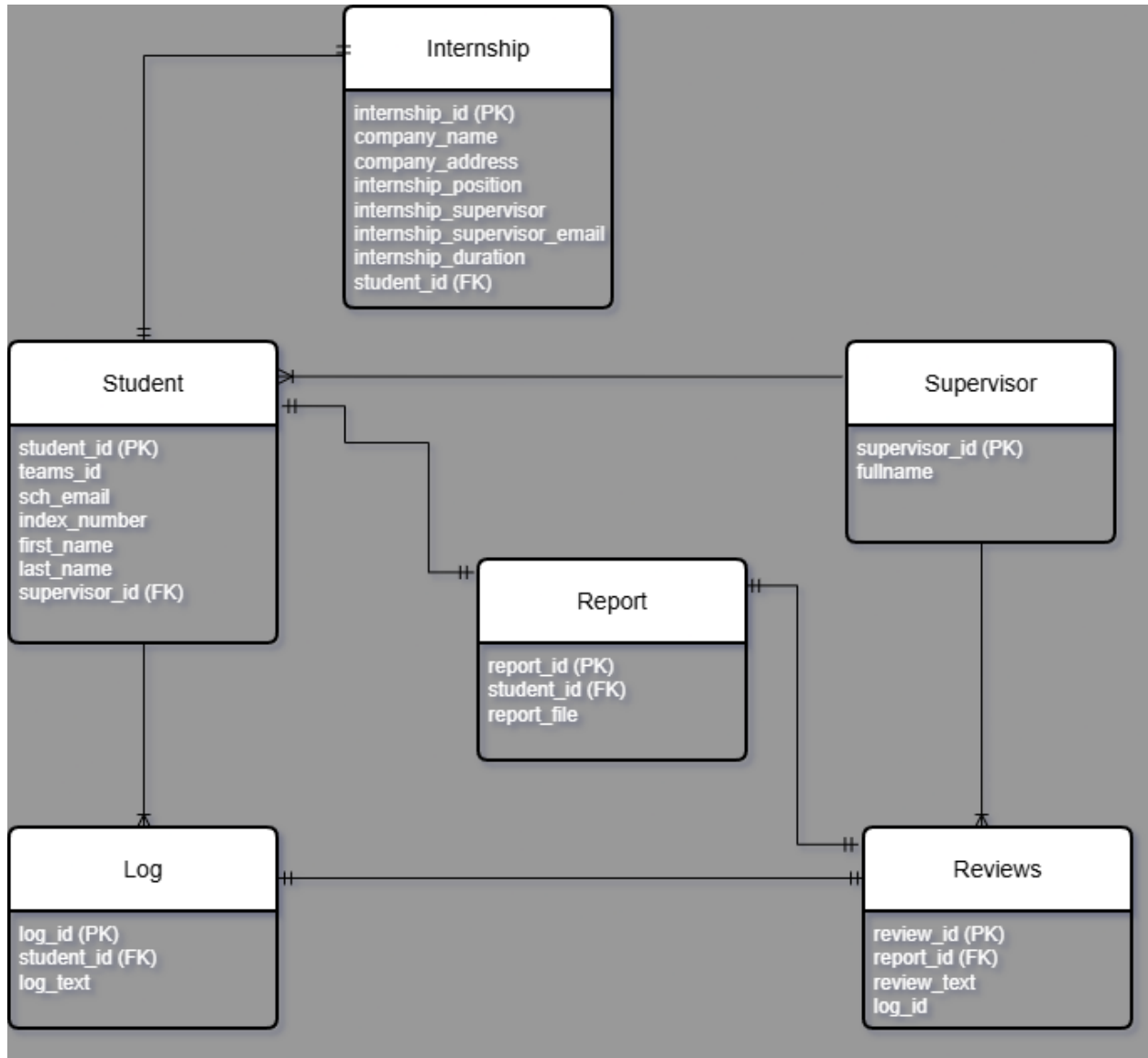


Figure 8. Entity Relationship Diagram

3.6.3 Users and Services

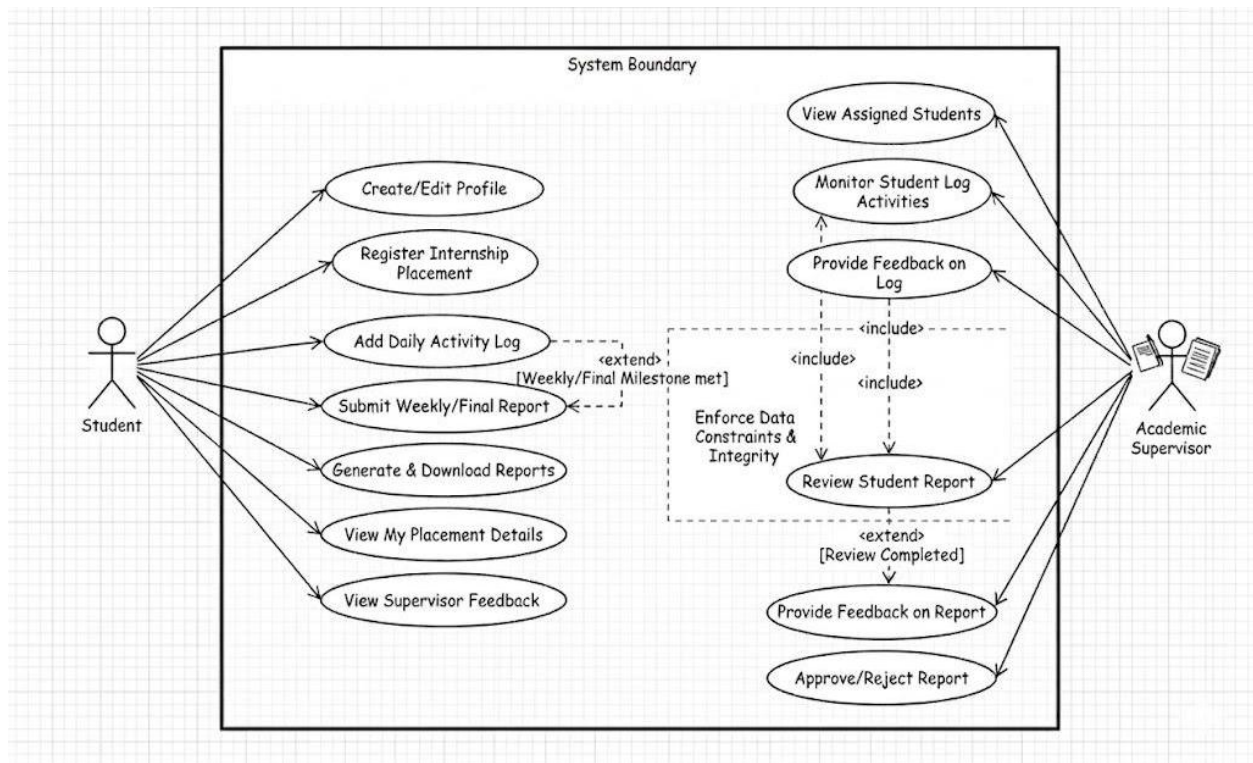


Figure 9. Use-Case Diagram

3.6.3.1 The Use Case Scenarios

USERS

This is the use case diagram of the internship and report management system. This platform uses different roles to determine the actions of its users. This platform consists of two roles. They are:

Student Role: This user has the permissions to perform these actions as follows:

- The student can create and edit their profile based on their preference.
- The student can register their place of internship.
- The student can upload their daily log activities.
- After the student has uploaded all their log activities, they can proceed to generate and download their report

- They are able to view feedback from their supervisors on their uploaded logs.

Supervisor Role: This is the supervisor role and they are granted the following permissions:

- The supervisor can view the total number of students assigned to them.
- They are able to monitor student log activities and provide feedback when necessary.
- They can also review the final report of students, provide feedback and decide whether to accept or reject the report.

3.6.3.2 *Services*

- Backend service: The backend is the most important component of the whole project. It acts as the core by processing user requests, handling user authentication and authorization for role-based access control. It also communicates with the database to store and retrieve data. Its API service helps ensure that the frontend can communicate with the backend without any issues.

3.6.4 Dependencies and Languages

3.6.4.1 *Dependencies*

- React

The frontend of this project was built using react which is a JavaScript library for building interactive, robust user interfaces and it is easy to set up.

Advantages

- It is suitable for both small and large-scale applications.
- It is compatible with a lot of modern tools like Vite and Tailwind CSS.

- Vite

Vite is a JavaScript tool that provide support for common web patterns with flexibility and it is optimized for react application to improve application performance.[18]

- Tailwind CSS

Tailwind CSS is an open-source, utility-first framework used for designing modern web applications directly in HTML using pre-defined classes. It speeds up UI development by reducing the need to write custom CSS files. It perfectly with JavaScript frameworks like React.[19]

- PostgreSQL

PostgreSQL is a powerful and open-source relational database system known for its reliability and data integrity. It organizes data into tables with rows and columns but extends traditional relational capabilities by supporting object-oriented features, such as custom data types, inheritance, and complex functions. which make it a good choice for small and big-scale applications.[20]

- Django

Django is an open-source python web framework designed for fast development of secure and scalable websites. It provides built in tools that are used for common tasks like authentication, an automated admin interface and URL routing. It works perfectly with major relational databases like PostgreSQL and MySQL. It is adopted by modern social media platforms like YouTube, Instagram and Pinterest due to its security features and the ability to handle complex data structures.[21]

3.6.4.2 Languages

- Python

Python is a high-level, open source and a general-purpose language for its simplicity using significant indentation and plain English syntax. It supports multi programming models like object-oriented programming.

Advantages

- It is easy to read and understand.
 - It can easily run on other operating systems.
 - It runs smoothly with other languages like C and C++ for complex tasks.
-
- JavaScript

JavaScript is a high-level programming language developed by Brendan Eich in 1995 alongside HTML and CSS. It forms the foundation of web development, enabling interactive features on websites such as animations, user interface controls, and real-time content updates. JavaScript is primarily a client-side language that runs directly in web browsers, but it also supports server-side development through runtimes like Node.js and can be used for mobile and desktop applications via frameworks like React Native. JavaScript is versatile because it can be used for both frontend and backend development. It is supported by all modern web browsers, allowing consistent performance across mobile, desktop and smart devices.

3.7 THE USER INTERFACE

Login

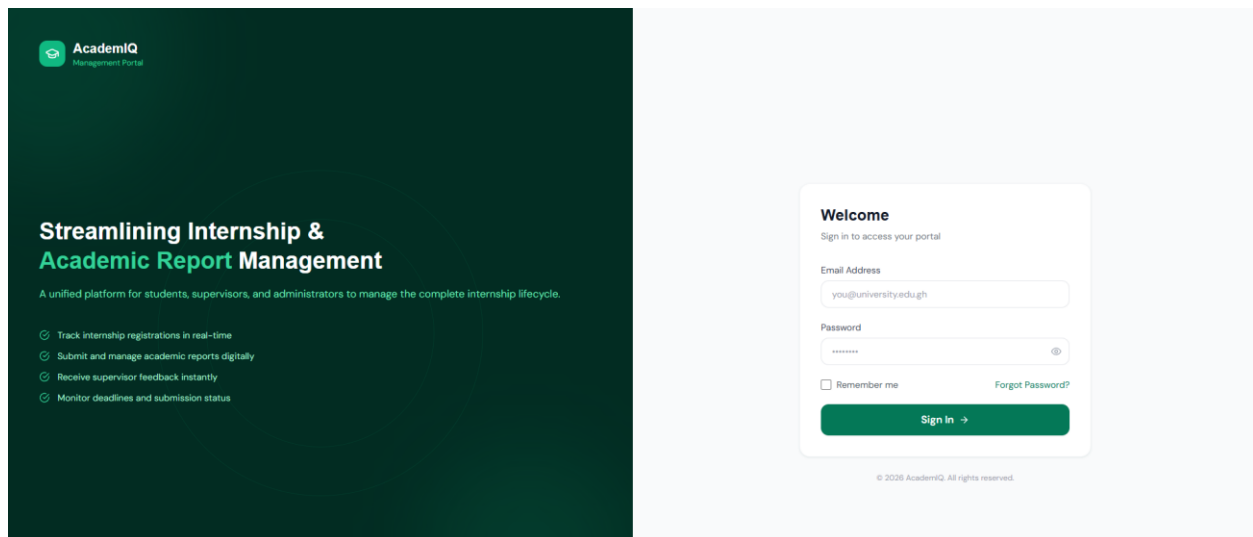


Figure 10. Login Page

The image above shows the login page where the user is authenticated. A user is required to have an email address and a password to login.

Student Dashboard

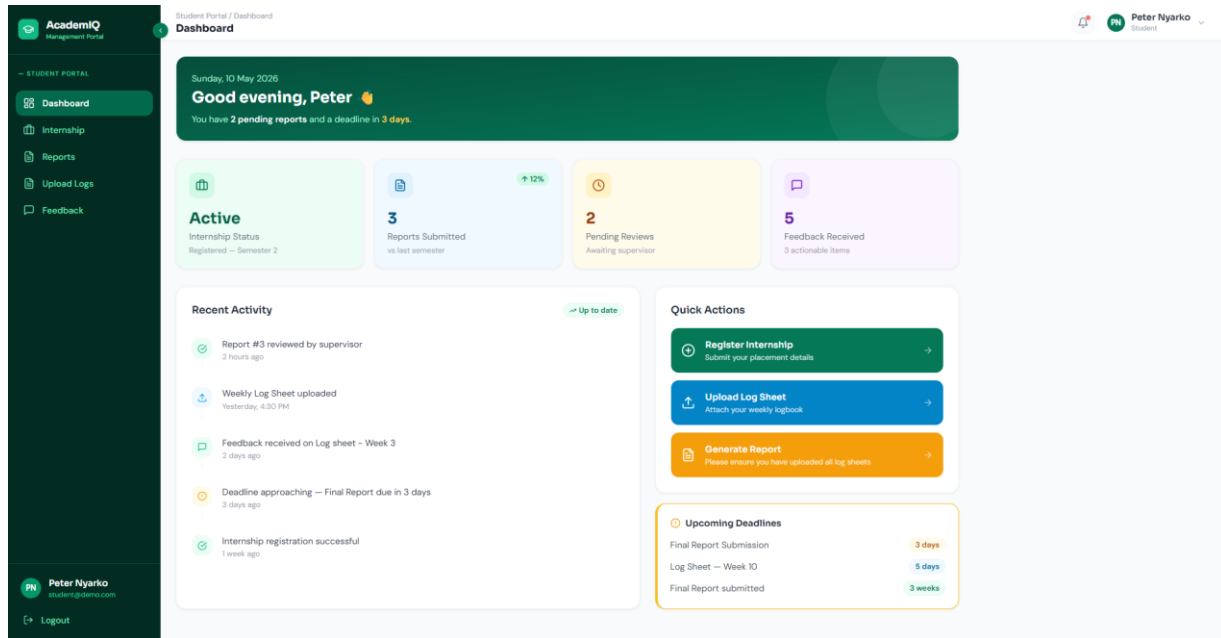


Figure 11. Student Dashboard

The figure above is the student Dashboard. It shows the summary of all the activities in the web application.

Internship Registration Page

Figure 12. Internship Registration

The image above shows the internship registration page. This section is where students are supposed to register their place of internship.

Upload Report

AcademiQ Management Portal

Student Portal / Reports / Upload

Upload Report

Peter Nyarko Student

STUDENT PORTAL

- Dashboard
- Internship
- Reports
- Upload Logs
- Feedback

Document File *

Drag & drop your weekly logs
or [browse files](#)
Accepts PDF, DOCX - Max 10MB

Log Report Title *

e.g. Industrial Training - Week 8

Report Type

Select report type..

Description (Mandatory)

Brief description of what this report covers..

Submit Report

Peter Nyarko
student@demo.com

Logout

Figure 13. Upload Weekly Logs

The figure above shows the upload logs section. In this section, the student can upload weekly internship activities.

Feedback

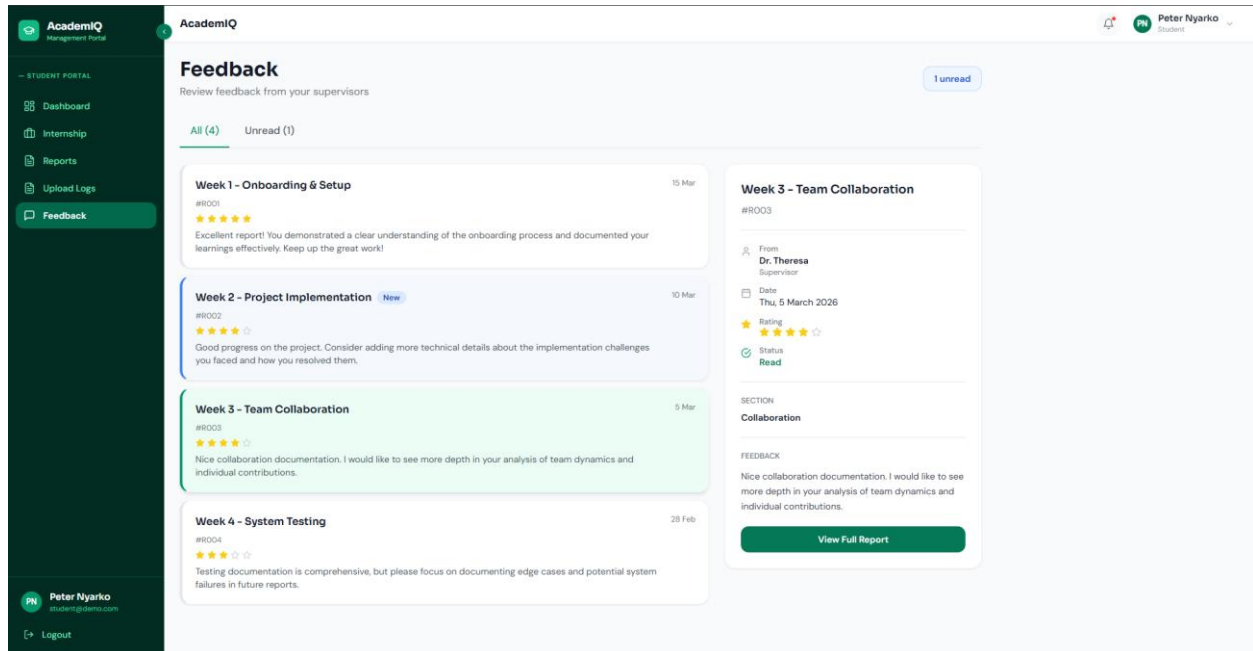


Figure 14. Feedback Page

The image above shows the feedback page. This is where students can see the reviews made by their supervisors.

Profile

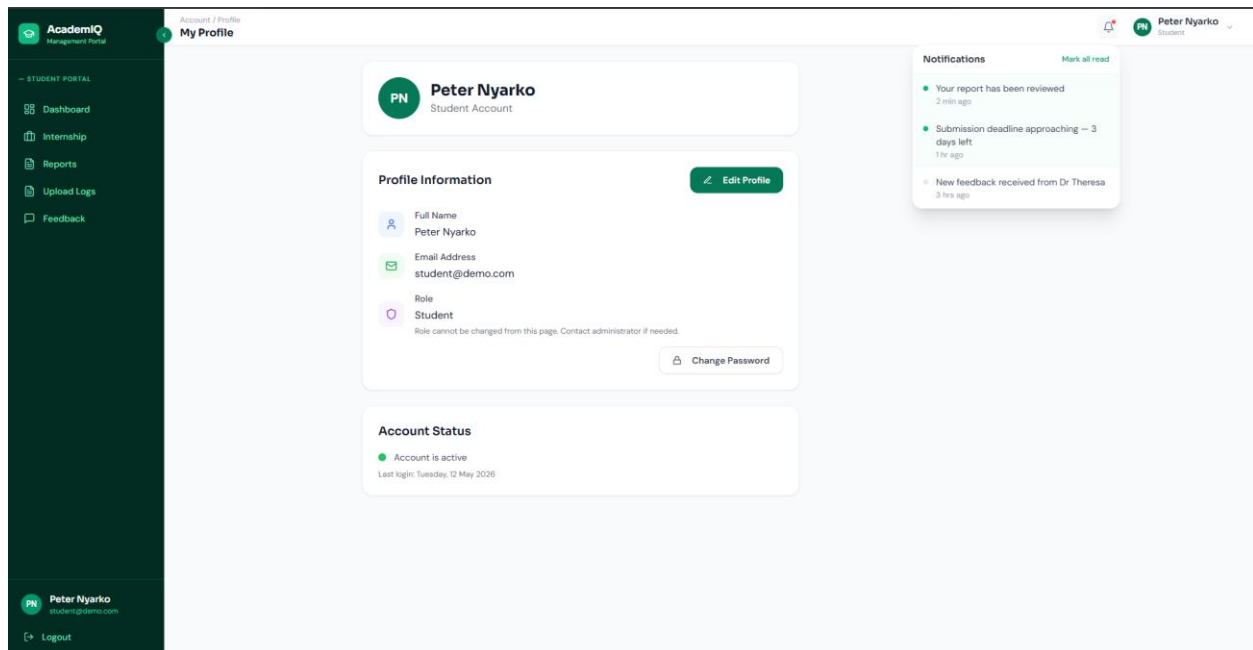


Figure 15. Profile Page

The figure above is the profile page. On this page, the user can change his or her details.

Supervisor Dashboard

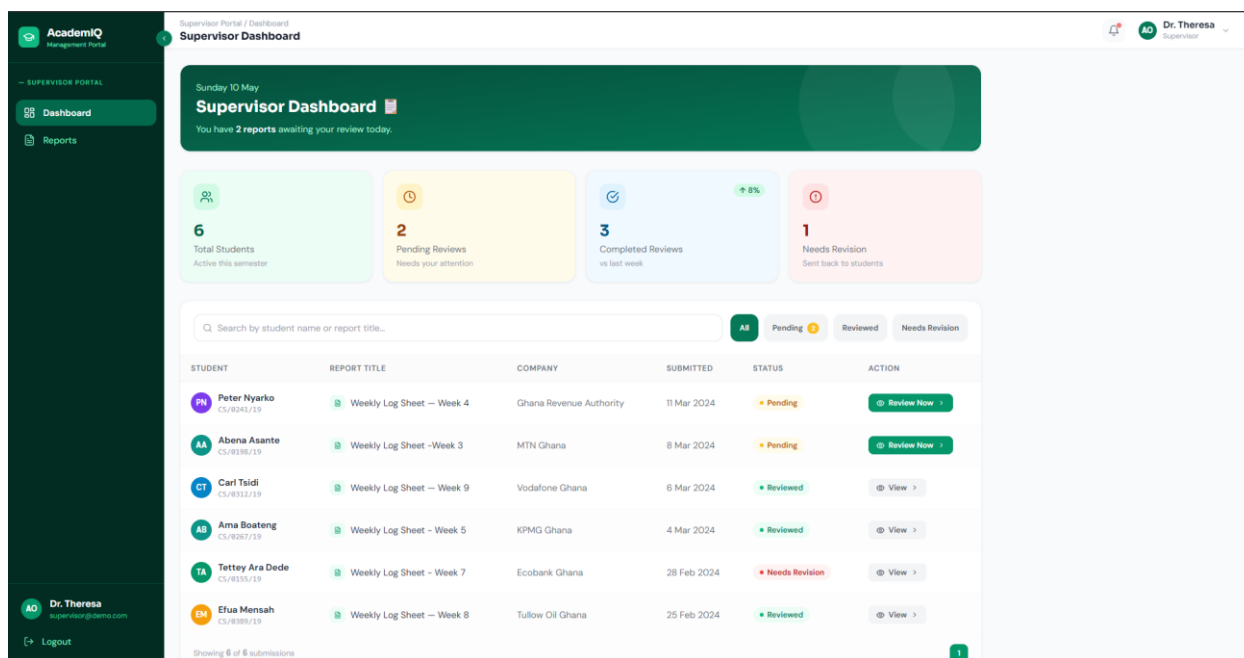


Figure 16. Supervisor Dashboard

This image above shows the supervisor dashboard. It indicates the total number of students assigned to the supervisor and students submitted log activities waiting for feedback.

Weekly Reports

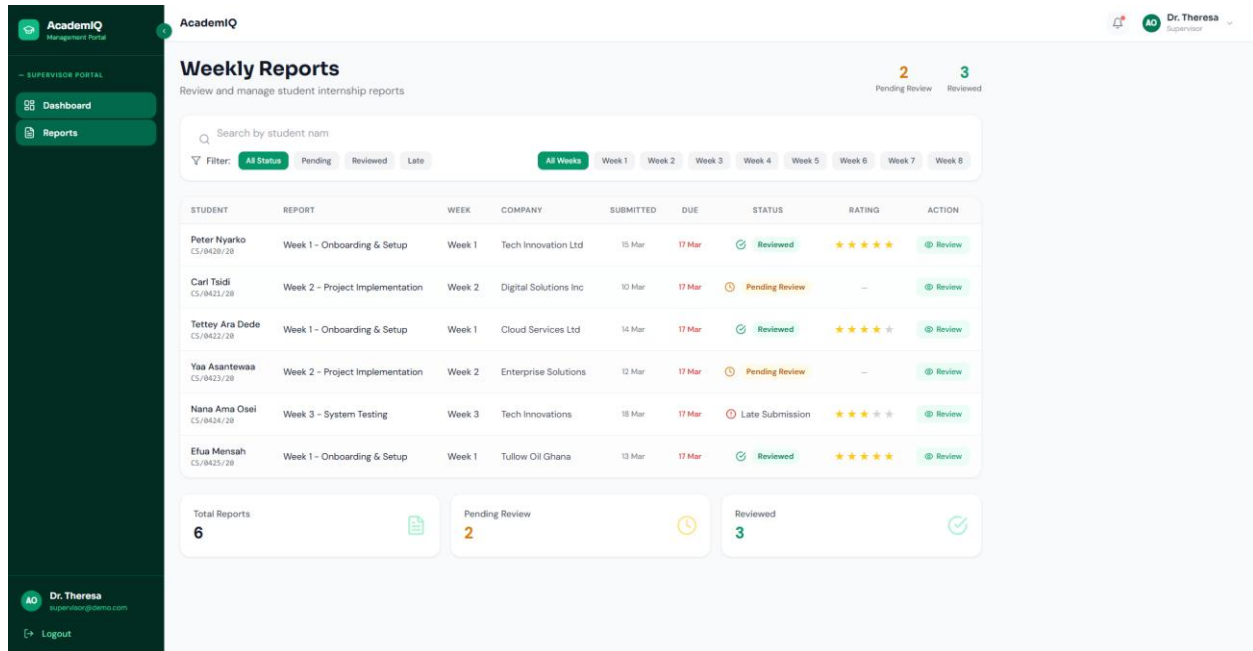


Figure 17. Supervisor's Weekly Reports Page

The figure above is the weekly reports page. This is where all the submitted logs of students are displayed.

Supervisor's Feedback

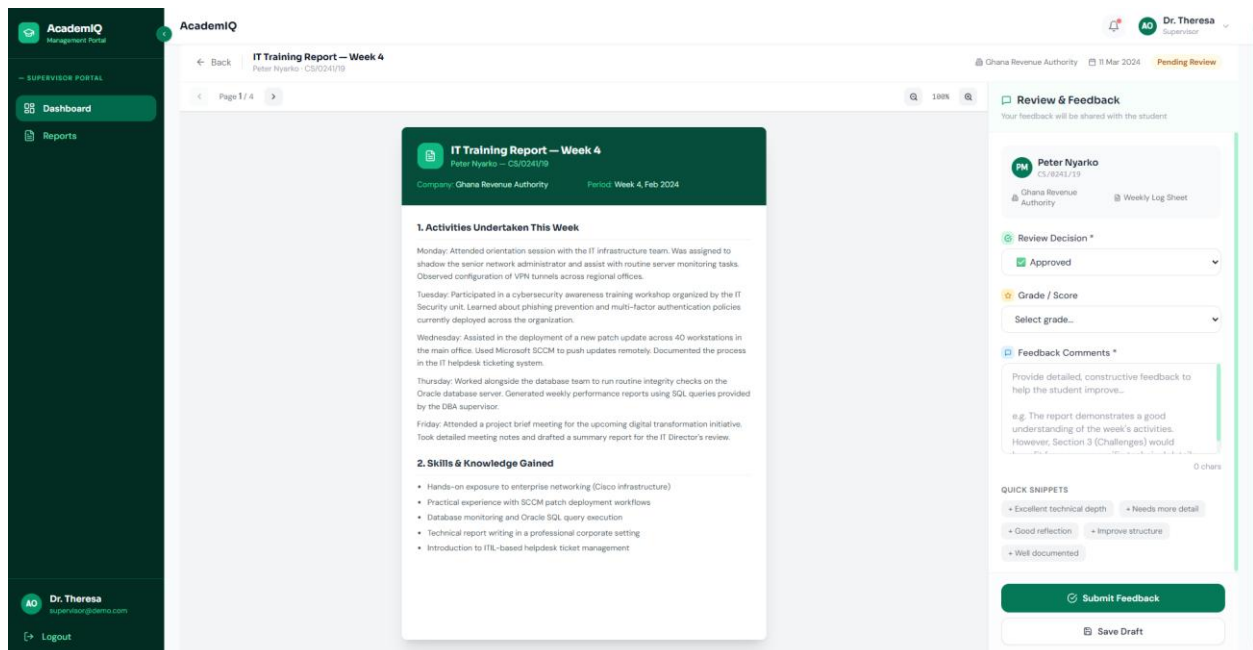


Figure 18. Supervisor's feedback page

The image above shows the supervisor feedback page. In this section, the supervisor is able to give feedback on the submitted logs of students.

4 References

- [1] R. Bosu, "The Role and Use of ICT in Administrative Activities in Higher Education Institutions: The Views of Administrators in Higher Educational Institutions in Ghana," *Academia*, December 2019. [Online]. Available: https://www.academia.edu/50461656/The_Role_and_Use_of_ICT_in_Administrative_Activities_in_Higher_Education_Institutions_The_Views_of_Administrators_in_Higher_Educational_Institutions_in_Ghana. [Accessed May 2026].
- [2] B. Makaa, "Importance of Industrial attachment/internships for students or interns," *Enhanced Education Group*, 6 March 2018. [Online]. Available: <https://www.eeducationgroup.com/importance-of-industrial-attachmentinternships-for-students-or-interns/?currency=KES>. [Accessed May 2026].
- [3] Quality Assurance and Planning Unit, Kwame Nkrumah University of Science and Tecnology, "KNUST Industrial Attachment and Internship Policy," Kwame Nkrumah University of Science and Technology, Kumasi, 2018.
- [4] Inplace Smart Placement Solutions, "Work Integrated Learning Placement Software," <https://inplacesoftware.com/higher-education/work-integrated-learning/>
- [5] PebblePad Ltd., "PebblePad Workbook and Portfolio Features," *PebblePad Documentation*, 2024.
- [6] A. Vaswani *et al.*, "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [7] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.
- [8] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022.
- [9] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] Lee Boonstra, "Prompt Engineering," 2025-01-18-pdf-1-TechAI-Goolge-whitepaper_Prompt Engineering_v4-af36d
- [11] OpenAI, "OpenAI API Documentation," 2024. [Online]. Available: <https://platform.openai.com/docs>
- [12] Ziwei Ji *et al.*, "Survey of Hallucination in Natural Language Generation" in *ACM Computing Surveys* (vol. 55, no. 12, pp. 1–38).

- [13] Ivana Vidakovic., “Notion AI vs Jasper.ai - Which One is Better For You?” [Online]. Available: <https://textcortex.com/post/notion-ai-vs-jasper>
- [14] M. Sami, “Software Development Life Cycle Models and Methodologies - Mohamed Sami,” Melsatar, 2012. [Online]. Available: <https://melsatar.blog/2012/03/15/software-development-life-cycle-models-and-methodologies/>.
- [15] **Beck**, K. et al. (2001). Manifesto for Agile Software Development. Agile Alliance. <https://agilemanifesto.org>
- [16] **Sommerville**, I. (2016). Software Engineering (10th ed.). Pearson Education.
- [17] <https://www.geeksforgeeks.org/system-design/client-server-architecture-system-design/>
- [18] Vite. (2024). Vite Official Documentation — Why Vite? <https://vitejs.dev/guide/why.html>
- [19] Tailwind CSS. (2024). Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- [20] What is Postgresql? <https://cloud.google.com/discover/what-is-postgresql>
- [21] <https://www.articsledge.com/post/django>